Proactive Remediations at Scale — Detection & Remediation Script Packs for Microsoft Intune

Implementation Solution Guide

Version 1.0.0 · 2026-07-21

Contents

Executive Summary — Proactive Remediations at Scale (Detection & Remediation Script Packs for Microsoft Intune) 4

1. Solution Overview — Proactive Remediations at Scale (Detection & Remediation Script Packs for Microsoft Intune) 4

- What this solution is . . . 4
- How it works — the detect/remediate model . . . 5
- Why safety is the whole game . . . 5
- The governed script-pack library concept . . . 6
- The standard deployment model . . . 6
- Benefits . . . 6
- Limitations (be honest with buyers) . . . 7

2. Requirements — Proactive Remediations at Scale (Detection & Remediation Script Packs for Microsoft Intune) 7

- R1. Proactive Remediations licensing entitlement . . . 8
- R2. Endpoint Analytics enabled (the parent solution) . . . 8
- R3. Windows devices enrolled and managed . . . 8
- R4. Intune Management Extension present . . . 8
- R5. PowerShell execution and script-signing posture . . . 9
- R6. Run-context permissions (system vs. user) . . . 9
- R7. A pilot device group . . . 9
- R8. A production target group . . . 9
- R9. An admin workstation with the Microsoft Graph PowerShell SDK . . . 9
- R10. A script-pack library location (governance) . . . 10

Global Requirements — Intune Solutions (category-level) 10

- G1. Microsoft Entra ID tenant . . . 10
- G2. Microsoft Intune subscription . . . 10
- G3. Administrative roles . . . 10
- G4. Device enrollment in Intune . . . 10
- G5. Pilot security group . . . 11
- G6. Supported, current operating system . . . 11
- G7. Network access to required cloud endpoints . . . 11

3. Implementation — Proactive Remediations at Scale (Detection & Remediation Script Packs for Microsoft Intune) 11

- Phase 1 — Author the pair (safety first) . . . 12
- Phase 2 — Package and scope . . . 13
- Phase 3 — Read results and validate . . . 14

Phase 4 — Expand to production behind guardrails . . . 15
Phase 5 — Govern the library . . . 15
5. Testing & Validation — Proactive Remediations at Scale (Detection & Remediation Script Packs for Microsoft Intune) 16
Test 1 — Package is present, assigned, and reporting . . . 16
Test 2 — Detection correctly flags the seeded issue (and only it) . . . 16
Test 3 — A bad remediation is caught in dry-run (preview) — before it can act . . . 16
Test 4 — Remediation fixes the seeded issue . . . 17
Test 5 — Re-detection proves the fix held . . . 17
Test 6 — Idempotency (no thrash on healthy devices) . . . 17
Acceptance gate . . . 17
6. Troubleshooting — Proactive Remediations at Scale (Detection & Remediation Script Packs for Microsoft Intune) 17
The script doesn't run at all (no status reported) . . . 17
The script runs but "does nothing" (wrong run context) . . . 18
Exit code misread as failure (or success) . . . 18
Remediation loops or over-applies (fix won't hold / churns the fleet) . . . 18
Output is truncated or missing in the console . . . 19
The Remediations experience is unavailable (licensing/entitlement gap) . . . 19
Useful checkpoints . . . 19
7. Best Practices — Proactive Remediations at Scale (Detection & Remediation Script Packs for Microsoft Intune) 19
Authoring the pair . . . 19
Deploy & scope . . . 20
Operate safely . . . 20
Govern the library . . . 21
8. Deliverables Summary — Proactive Remediations at Scale (Detection & Remediation Script Packs for Microsoft Intune) 21
Documentation (this guide) . . . 21
PowerShell scripts (Microsoft Graph & templates) — `PROREM-Scripts.zip` . . . 22
Hero image . . . 22
How it's delivered . . . 22
Appendix A — PowerShell Scripts 23

Executive Summary — Proactive Remediations at Scale (Detection & Remediation Script Packs for Microsoft Intune)

The same endpoint problems come back across thousands of devices — a stopped service, a drifted registry value, an expired client certificate, a setting that keeps reverting. Fixed reactively, one ticket at a time, they never end: the help desk burns hours on repeats, and users hit the same issue again next week.

Proactive Remediations — part of **Endpoint Analytics** in Microsoft Intune — breaks that cycle. A **detection** script runs on a schedule and decides *"is this device affected?"*; a **remediation** script runs **only on the devices that are** to fix it — automatically, fleet-wide, with reporting. This solution is the **engine and the discipline** behind that capability:

- **Author safe pairs** → idempotent, read-only **detection** with a strict exit-code contract, and

minimal remediation that changes the smallest possible thing — proven in a **dry-run/preview** before it is ever allowed to act.

- **Deploy, scope, and schedule** → package the pair as a named Intune remediation, pilot it to a

small ring, and read the detection/remediation status and output the console reports.

- **Prove it worked** → **re-detection** confirms the fix held, and a success/failure trend shows

"with issue" dropping toward zero.

- **Govern a reusable library** → versioning, naming, and second-admin review/approval keep a growing

set of proven script packs trustworthy, so each solved problem makes the next one faster.

The through-line is **safety**: idempotency, dry-run-first, exit-code discipline, correct run context, pilot rings, and rollback — the guardrails that keep auto-remediation from becoming **auto-outage**. This package plugs into the Endpoint Analytics you have already enabled (see the companion deployment solution); it does not re-do that onboarding.

Built the standard repeatable way — **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** — with exact, click-by-click Intune steps, a requirements guide that tells you how to verify *and* fix each prerequisite, five ready-to-run PowerShell scripts (three read-only Graph tools plus a detection and a preview-first remediation template), and testing/troubleshooting guides with clear pass/fail gates.

Outcome: a working Proactive Remediations practice that finds and fixes recurring endpoint issues fleet-wide **before users notice** — operated safely by your own team, without a consulting engagement.

1. Solution Overview — Proactive Remediations at Scale (Detection & Remediation Script Packs for Microsoft Intune)

What this solution is

This solution turns Microsoft Intune's **Proactive Remediations** engine (part of **Endpoint Analytics**) into a disciplined, fleet-wide **detect-and-fix practice**. Instead of fixing the same recurring endpoint problem one help-desk ticket at a time — a stopped service, a drifted registry value, a stale scheduled task, a missing setting — you deploy a **script pair**: a **detection** script that runs on a schedule and decides *"is this device

affected?"*, and a **remediation** script that runs **only on the devices that are** to put them right. The result is reported back to the console as a success/failure trend, so you can *prove* the fix landed and held.

The heart of this package is not just "how to click Create" — it is **how to author detect/remediate PowerShell pairs that are safe to run unattended on thousands of machines**, and the **governance** that keeps a reusable library of those pairs trustworthy over time. Automated remediation is powerful; done carelessly it is an **auto-outage generator**. This solution goes deep on the safety discipline — idempotent detection, minimal remediation, a **dry-run/preview default**, a strict **exit-code contract**, correct **run context** (system vs. user), logging, rollback thinking, and **pilot rings** — so the automation only ever helps.

> ***Where this sits.*** *Proactive Remediations lives inside* ***Endpoint Analytics****. The companion solution* ***"Endpoint Analytics Deployment & Optimization in Microsoft Intune"*** *covers turning Endpoint Analytics *on* and reading its Startup / Reliability / Work-From-Anywhere scores — this package is the* ***remediation engine that plugs into it*** *and assumes Endpoint Analytics is already enabled. This document does* ***not*** *re-teach Endpoint Analytics onboarding; see that parent solution for it (referenced, not reproduced).*

How it works — the detect/remediate model

1. **You author two scripts.** A **detection** script inspects the device for one specific,

well-defined condition and signals the result **only through its exit code** — a non-zero exit means *"issue found, run the remediation"*; an exit of zero means *"healthy, do nothing."*

1. **Intune runs detection on a schedule.** You package the pair as a **remediation** (the console

object) and assign it to a group of devices with a recurrence (for example, once daily). The detection runs on every targeted device on that cadence.

1. **Remediation runs only where needed.** On any device whose detection exited non-zero, Intune

runs the paired **remediation** script to apply the fix. Devices that were already healthy are never touched.

1. **Re-detection proves the fix.** On the next scheduled cycle, detection runs again; a device that

was remediated should now report healthy, which is your evidence the fix worked and **held**.

1. **The console reports the trend.** Endpoint Analytics shows per-remediation columns —

detection status, remediation status, and the scripts' output — aggregated across the fleet so you can watch "with issue" shrink toward zero.

Why safety is the whole game

A detection script that runs read-only is low-risk. A **remediation** script *changes state on every affected machine at once* — so a subtle bug doesn't cause one bad outcome, it causes **thousands, simultaneously**. The discipline that prevents that is the core of this solution:

• **Idempotent detection** — detection must reach the *same* verdict every run and must not itself

change the device. It only *observes* and reports via exit code.

• **Minimal remediation** — the fix changes the **smallest possible** thing to resolve exactly the

detected condition, then stops. No "while I'm here" extras.

- **Dry-run / preview first** — a remediation is built and proven in a **preview mode** (it logs *what it would change* without changing it) before it is ever allowed to act. The included remediation **template defaults to a preview** for exactly this reason.
- **Exit-code contract** — detection uses `exit 0` (healthy) / `exit 1` (issue) and nothing else to communicate; remediation uses `exit 0` to report success. Misreading this contract is the most common cause of "it says failed but the device is fine" (and vice-versa).
- **Correct run context** — you choose whether the pair runs as **system** (machine-wide, no user needed) or in the **logged-on user's** context, because a script targeting the user hive run as system silently does nothing.
- **Pilot rings + rollback** — every pair is proven on a **small pilot ring** before it widens, and you keep the ability to unassign or reverse it. Nothing goes fleet-wide unproven.

The governed script-pack library concept

Ad-hoc scripts pasted into the console rot: nobody knows which version is live, who approved it, or what it does. This solution treats your detect/remediate pairs as **governed, reusable assets** — a **script-pack library** with:

- **A naming convention** so every pack is identifiable at a glance (see Implementation).
- **Versioning** — each pack carries a version and a changelog, so you know exactly what is deployed.
- **Review & approval** — a remediation is reviewed by a second admin and proven in preview + pilot before it is approved to act in production.
- **Reuse** — a proven pack (e.g. "restart a stopped service", "correct a registry value") is kept and re-applied to the next recurring issue, so the practice compounds instead of restarting.

The standard deployment model

This solution uses **one** repeatable model so the process is recognizable across the catalog:

> ***Author an idempotent detection script and a minimal, preview-first remediation script; package them as a named Intune remediation; scope to a small pilot ring; schedule; read the detection and remediation status/output; then wave to production behind guardrails — and register every proven pair in a versioned, reviewed script-pack library.***

Every step follows the repeatable pattern: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand.**

Benefits

- **Fix problems before users notice** — recurring issues are detected and remediated on a schedule, fleet-wide, instead of arriving as tickets.
- **Eliminate repetitive toil** — the same fix stops being a manual, per-device task; it becomes a

script pack that runs itself.

• **Safe by construction** — idempotent detection, minimal preview-first remediation, exit-code discipline, and pilot rings keep auto-remediation from becoming auto-outage.

• **Provable outcomes** — re-detection and the console's success/failure trend give you evidence the fix worked and held, not just a hope.

• **A compounding asset** — a governed, versioned library of reusable packs means each solved problem makes the next one faster.

• **Plugs into Endpoint Analytics** — reuses the analytics surface you already enabled; the scores tell you *where* to point a remediation next.

Limitations (be honest with buyers)

• **Licensing & availability gate the feature.** Proactive Remediations is part of Endpoint Analytics and requires specific licensing and enrollment prerequisites — **verify current licensing and feature availability against Microsoft's documentation** (Requirements §R1). It does not cover every platform.

• **Endpoint Analytics must be enabled first** — this package assumes the parent solution's onboarding is done; it does not re-do it.

• **Remediation power is a risk surface** — a poorly authored remediation can do harm at scale. The safety discipline in this guide is mandatory, not optional; it is not a substitute for reviewing your own scripts.

• **Scheduling is periodic, not instant** — detection runs on a recurrence, so remediation is "eventually", not real-time. Exact scheduling behavior and cadence limits should be **verified against current Microsoft documentation**.

• **Output visibility has limits** — the console captures script output within size limits and on a reporting delay; very large output is truncated (see Troubleshooting).

• **Not for app-install failures** — remediating failed application deployments is a different domain handled by a separate app-troubleshooting solution; this package is for endpoint state/config remediations, not app delivery.

2. Requirements — Proactive Remediations at Scale (Detection & Remediation Script Packs for Microsoft Intune)

Confirm **every** requirement below before you begin. Each entry follows the same pattern: **Description · Why required · How to verify · How to implement if missing.** Category-wide prerequisites (tenant, Intune licensing, admin roles, device enrollment, supported OS, network) are covered once in the **Global Requirements — Intune Solutions** document; the items here are specific to authoring, deploying, and operating Proactive Remediations script packs.

> ***Verify-with-Microsoft flag.*** *Proactive Remediations licensing, feature availability, run-context behavior, scheduling limits, and console navigation* ***change over time and differ by cloud (Commercial vs. GCC / GCC High / DoD)****. Where this document states a licensing SKU, a portal path, or a behavior, treat it as a starting point and* ***verify it against current Microsoft documentation for your tenant and cloud*** *before relying on it. Facts you must not assume are called out inline.*

R1. Proactive Remediations licensing entitlement

Description: A license that entitles the tenant to **Endpoint Analytics Proactive Remediations**. **Why:** Proactive Remediations is a licensed Endpoint Analytics capability; without the entitlement the **Devices** → **Remediations** experience is unavailable or blocked. **Verify:** In Intune, open **Reports** → **Endpoint analytics** → **Remediations** (path may differ by tenant — **verify against current Microsoft documentation**). If you can create a remediation, the entitlement is present. **Do not assume a specific SKU grants this — confirm the current licensing requirement with Microsoft's documentation** for your subscription and cloud (Commercial vs. GCC/GCC High/DoD may differ). **Implement:** Acquire/assign the licensing Microsoft currently requires for Endpoint Analytics Proactive Remediations (verify the exact SKU with Microsoft), then re-check the console.

R2. Endpoint Analytics enabled (the parent solution)

Description: Endpoint Analytics is **onboarded/enabled** in the tenant. **Why:** Proactive Remediations is a component of Endpoint Analytics; the remediation reporting surface lives inside it. This package is the **remediation engine**, not the onboarding. **Verify:** Intune → **Reports** → **Endpoint analytics** shows an active dashboard (scores are populating), not an "enable" prompt. **Implement:** Complete Endpoint Analytics onboarding first — see the companion solution **"Endpoint Analytics Deployment & Optimization in Microsoft Intune"** (referenced here, not reproduced). Return to this guide once analytics is live.

R3. Windows devices enrolled and managed

Description: The target devices are **Windows** devices enrolled in Intune (MDM-managed or co-managed with the workload directed to Intune, per Microsoft's current support statement). **Why:** Remediation script pairs are delivered to enrolled Windows endpoints via the Intune management extension; unenrolled devices cannot receive them. **Platform support and the exact co-management requirement should be verified against current Microsoft documentation. Verify:** Intune → **Devices** → **All devices**; targets appear, are **Windows**, and are compliant. **Implement:** Enroll the devices (per Global Requirements G4) and confirm they check in.

R4. Intune Management Extension present

Description: The **Intune Management Extension (IME)** agent is installed and healthy on targets. **Why:** IME is what actually runs PowerShell scripts and remediation script pairs on the device; if it is absent or unhealthy, detection/remediation never runs. **Verify:** On a target, confirm the **Microsoft Intune Management Extension** service is present and running; IME installs automatically when a device receives a PowerShell script or Win32 app assignment. (**Exact service name/behavior — verify against current Microsoft documentation.**) **Implement:** Assign any Intune PowerShell script or Win32 app to the device to trigger IME installation, then re-check.

R5. PowerShell execution and script-signing posture

Description: A decision on whether your remediation scripts are **signed**, and an execution policy posture that lets IME run them. **Why:** IME runs remediation scripts in a defined execution context; organizations that enforce script signing must sign their detect/remediate scripts or the run fails. **The exact signing/ execution behavior is set by Intune and your environment — verify against current Microsoft documentation. Verify:** Author a trivial detection that `exit 0`, deploy to one pilot device, and confirm it runs (status reports back). If it fails to run, signing/execution policy is the likely cause. **Implement:** Establish a code-signing process for your script packs (recommended for governance), or configure the remediation to run in the context Microsoft documents as not requiring signing — **verify current behavior before relying on either.**

R6. Run-context permissions (system vs. user)

Description: Clarity on whether each script pair must run as **system** (machine context, 64-bit, no signed-in user needed) or in the **logged-on user's** context. **Why:** A remediation that edits the machine registry/services must run as **system**; one that edits the current user's profile/HKCU must run in the **user** context. Choosing wrong means the script runs but **silently does nothing** to the intended target — a top cause of "it says success but nothing changed." **Verify:** For each pair, identify what it reads/writes (HKLM/service/machine file → system; HKCU/user profile → user) and match the remediation's **run context** setting to it. **Implement:** Set the correct **Run this script using the logged-on credentials** choice when you create the remediation (Implementation §5). Test on a device with a signed-in user for user-context pairs. **Confirm the current run-context options and their exact behavior against Microsoft's documentation.**

R7. A pilot device group

Description: A Microsoft Entra security group — **named** `PILOT – Remediations – Devices` — containing a small set of representative Windows devices. **Why:** No remediation goes fleet-wide unproven; the pilot ring is where you catch a bad pair before it can do harm at scale. **Verify:** Entra → **Groups**; `PILOT – Remediations – Devices` exists with a handful of pilot devices. **Implement:** Entra → **Groups** → **New group** → **Security**; name it `PILOT – Remediations – Devices` and add the pilot devices (Implementation §4).

R8. A production target group

Description: A Microsoft Entra security group — **named** `Remediations – Windows – Production` — representing the broader fleet you will eventually target. **Why:** After the pilot passes you widen to production in waves; a named production group is the assignment target for that expansion. **Verify:** Entra → **Groups**; `Remediations – Windows – Production` exists (it may start empty and fill in waves). **Implement:** Entra → **Groups** → **New group** → **Security**; name it `Remediations – Windows – Production`.

R9. An admin workstation with the Microsoft Graph PowerShell SDK

Description: A workstation with the **Microsoft.Graph** PowerShell SDK for the discovery, validation, and reporting scripts in this package. **Why:** The included read-only scripts query Intune via Microsoft Graph to

inventory remediations, their assignments, and run status. **Verify:** `Get-Module Microsoft.Graph -ListAvailable` returns the SDK; `Connect-MgGraph` succeeds. **Implement:** `Install-Module Microsoft.Graph -Scope CurrentUser`. The scripts request read scopes such as `DeviceManagementConfiguration.Read.All` and consent on first connect. (**Graph endpoints/ scopes for remediations may change — verify against current Microsoft documentation.**)

R10. A script-pack library location (governance)

Description: A version-controlled location (a repository or a controlled share) — referred to as `Remediations Pack Library` — where the source of every detect/remediate pair is stored, versioned, and reviewed before deployment. **Why:** The console alone is not a source of truth; governed packs (naming, versioning, approval) are what keep the practice safe and auditable over time. **Verify:** The `Remediations Pack Library` location exists and holds the current pair sources with version history. **Implement:** Create the repository/share (Implementation §11), adopt the naming and versioning convention, and require a second-admin review before a remediation is approved to act.

Global Requirements — Intune Solutions (category-level)

These requirements apply to **every** solution in the **Intune Solutions** category. They are published once at the category level on the marketplace so individual solution listings stay small. Confirm these are in place before deploying *any* Intune solution.

Each entry: **Description · Why required · How to verify · How to implement if missing.**

G1. Microsoft Entra ID tenant

Description: An active Microsoft Entra ID (Azure AD) tenant. **Why:** It is the identity backbone for every cloud and Intune-managed resource. **Verify:** Sign in to `https://entra.microsoft.com` as an administrator. **Implement:** Create a Microsoft 365/Azure subscription, which provisions a tenant.

G2. Microsoft Intune subscription

Description: An Intune-bearing license plan. **Why:** Intune delivers all configuration, compliance, enrollment, and security policies. **Verify:** `https://intune.microsoft.com` loads and **Tenant administration** → **Tenant status** shows an active subscription. **Implement:** Assign Microsoft 365 E3/E5, Business Premium, or standalone Intune Plan 1.

G3. Administrative roles

Description: Appropriate Entra/Intune admin roles for the operator. **Why:** Creating policies, enrollment connectors, and configuration profiles requires elevated rights. **Verify:** Entra → **Roles and administrators**; confirm **Intune Administrator** (or Global Administrator for connector setup). **Implement:** Assign **Intune Administrator** to the operator; reserve Global Administrator for tenant-wide changes.

G4. Device enrollment in Intune

Description: Targeted devices are enrolled and manageable in Intune. **Why:** Policies only reach enrolled (or app-managed) devices, regardless of platform. **Verify:** Intune → **Devices** → **All devices**; the device

appears and is **compliant**. **Implement:** Configure the appropriate enrollment method for the platform and ownership model, then enroll the device.

G5. Pilot security group

Description: A Microsoft Entra security group for pilot/targeted users or devices. **Why:** Every solution is piloted to a small group before fleet rollout. **Verify:** Entra → **Groups**; the pilot group exists with intended members. **Implement:** Entra → **Groups** → **New group** → type **Security** → add pilot members.

G6. Supported, current operating system

Description: A supported OS baseline with current updates for the targeted platform. **Why:** Modern Intune features and security settings assume a patched, vendor-supported OS. **Verify:** Check the device's OS version or Intune → device → **Hardware**. **Implement:** Deploy update policies or upgrade to a supported release.

G7. Network access to required cloud endpoints

Description: Outbound HTTPS to Microsoft 365, Entra, and Intune service URLs. **Why:** Enrollment, policy sync, and authentication depend on these endpoints. **Verify:** From a device, reach `https://login.microsoftonline.com` and `https://enrollment.manage.microsoft.com`. **Implement:** Allow Microsoft's published Office 365/Intune URL & IP ranges through proxy/firewall.

The standard deployment model (all solutions): Assess → Configure → Pilot → Validate → Expand.

3. Implementation — Proactive Remediations at Scale (Detection & Remediation Script Packs for Microsoft Intune)

Follow these steps **in order**. The *why* lives in the Solution Overview; the steps below are short and exact so you can **click to implement**. The design is: **author an idempotent detection script and a minimal, preview-first remediation script → package them as a named Intune remediation → scope to a pilot ring → schedule → read status/output → wave to production behind guardrails → register the proven pair in a governed library.** Pilot first: **Assess → Configure → Pilot → Validate → Expand.**

Consoles used:

- **Microsoft Intune admin center** — `https://intune.microsoft.com`
- **Microsoft Entra admin center** — `https://entra.microsoft.com`

Navigation flag. *The exact **Remediations** location and blade names in Intune change over time and differ by cloud (Commercial vs. GCC / GCC High / DoD). This guide uses the common path **Reports** → **Endpoint analytics** → **Remediations** (some tenants also surface it under **Devices** → **Remediations**). **Verify the current path against Microsoft's documentation for your tenant** before creating objects.*

Worked example used throughout. *So every step is concrete, this guide builds one real pair: a **service-health** remediation that ensures the **Windows Time service (**`w32time`**)** is set to **Automatic** and **Running** — a common recurring drift. Reuse the *pattern* (and the included templates) for your*

own conditions; the object names below are the ones to reuse verbatim.

Phase 1 — Author the pair (safety first)

Step 1. Author the detection script (idempotent, read-only, exit-code contract)

1. In your `Remediations Pack Library` location (Requirements §R10), create a file **named `Detect-ServiceHealth-w32time.ps1`**. Use the included `PROREM-Detect-Template.ps1` as the starting point.

1. The detection must do **three things and nothing else**:

• **Observe only** — read the state (here: query the `w32time` service start type and status). It must **never change** the device. Detection is read-only by contract.

• **Reach the same verdict every run** (idempotent) — given the same device state, it always returns the same result.

• **Report via exit code only** — `exit 0` = **healthy** (do nothing); `exit 1` = **issue found** (run the remediation). Write a short human-readable line to output for the console, but the **exit code is the contract**.

1. Example logic (conceptual): if the service is `Automatic` **and** `Running` → `Write-Output "w32time healthy"; exit 0. Otherwise` → `Write-Output "w32time not Automatic/Running"; exit 1`.

1. Save it. **Do not** put any fix logic in the detection script — a detection that "fixes while it checks" breaks the model and hides what remediation actually did.

Step 2. Author the remediation script (minimal, preview-first, logged)

1. In the same library location, create a file **named** `Remediate-ServiceHealth-w32time.ps1`. Use the included `PROREM-Remediate-Template.ps1` as the starting point — **it defaults to a preview (dry-run)**.

1. The remediation must:

• **Default to preview** — on first authoring and first pilot, it **logs exactly what it would change without changing it**. You only remove the preview gate once you have proven, from the log, that it would do the right and minimal thing. (The template's `-WhatIf`-style `$DryRun = $true` default enforces this.)

• **Change the smallest possible thing** — set the `w32time` start type to `Automatic` and start the service if stopped. Nothing else. No "while I'm here" extras.

• **Be idempotent** — running it on an already-fixed device makes no further change and still `exit 0`.

• **Log to a predictable path** — write a timestamped line to a log (e.g. under

`C:\ProgramData\Remediations\`) so you have on-device evidence of every action.

• **Report success via** `exit 0` — a remediation that completes its fix exits `0`; if it cannot

fix (e.g. insufficient rights, target absent), it writes the reason and `exit 1`.

1. **Prove it in preview first:** run the remediation locally (or on one pilot device) with the

dry-run default and read the log — confirm it *would* make exactly the intended minimal change. Only then produce the "acting" version by setting the dry-run switch off for deployment.

Step 3. Match the run context to what the pair touches

1. Decide the **run context** for this pair. The `w32time` service is **machine-wide**, so this pair

runs as **system** (no signed-in user required). A pair that edited `HKCU`/the user profile would instead run in the **logged-on user's context** (Requirements §R6).

1. Record the chosen context in the pack's header/changelog so reviewers see it. A mismatch here is

the classic "runs but changes nothing" failure.

Phase 2 — Package and scope

Step 4. Create the pilot and production groups

1. Entra → **Groups** → **New group** → **Security**; **name it** `PILOT – Remediations – Devices` and add a

small set of representative Windows devices.

1. Entra → **Groups** → **New group** → **Security**; **name it** `Remediations – Windows – Production` (it

may start empty; you fill it in waves during Expand).

Step 5. Create the remediation (package the pair) — name it

1. In Intune, go to **Reports** → **Endpoint analytics** → **Remediations** → **Create script package**

(*verify the current path per the navigation flag above*).

1. **Basics: name it** `PROREM – Service Health – w32time – Pilot`; add a clear description and

the **publisher** (your team name — white-label; do not brand it).

1. **Settings:**

• **Detection script file:** upload `Detect-ServiceHealth-w32time.ps1` (Step 1).

• **Remediation script file:** upload `Remediate-ServiceHealth-w32time.ps1` (Step 2) — for the

pilot, upload the **preview/dry-run** version first so the pilot proves behavior without changing devices.

• **Run this script using the logged-on credentials:** set to **No** for this system-context pair

(set **Yes** only for user-context pairs — Step 3).

• **Enforce script signature check** and **Run script in 64-bit PowerShell:** set per your

signing posture (Requirements §R5) and the target's needs — a script touching 64-bit locations must run 64-bit. (**Verify current option names/behavior against Microsoft's documentation.**)

1. **Review + create.**

Step 6. Assign to the pilot ring and schedule

1. Open `PROREM – Service Health – w32time – Pilot` → **Assignments** → **Select groups to include**

→ choose `PILOT – Remediations – Devices`.

1. Set the **schedule** for the assignment. Start with **Once** (or **Daily**) for the pilot so you

see a clean first result quickly. (Available cadences — Once / Hourly / Daily — and their exact behavior should be **verified against current Microsoft documentation**.)

1. **Save.** Detection now runs on the pilot devices on that schedule; because the pilot uses the

preview remediation, no device is changed yet — you are validating detection accuracy and the remediation's *intended* behavior from its log.

Phase 3 — Read results and validate

Step 7. Read detection and remediation status + output

1. Open `PROREM – Service Health – w32time – Pilot` → the **Overview / Device status** view.
2. Read the columns Endpoint Analytics reports (names may vary — **verify current column labels**):

• **Detection status** — e.g. *With issue* / *Without issue* / *Detection failed*.

• **Remediation status** — e.g. *Issue remediated* / *Issue not remediated* / *Remediation

failed* (in preview, the remediation reports what it *would* have done via its output/log).

• **Pre-remediation** and **post-remediation** detection output — the human-readable lines your

scripts wrote.

1. Confirm the detection correctly split the pilot into *With issue* vs *Without issue*, and that the

With issue devices' remediation **preview log** shows the correct minimal intended change.

1. Run the included `PROREM-Discovery.ps1` and `PROREM-Validate.ps1` for a tenant-side view

(the package exists, is assigned, is running, and is reporting).

Step 8. Flip the pilot remediation from preview to acting, and re-detect

1. Once the preview logs prove correctness, update the remediation script to its **acting** version

(dry-run off) and re-upload it to `PROREM – Service Health – w32time – Pilot` (**Properties** → **Settings** → **Remediation script file** → **replace**).

1. Let the schedule run again. Now the *With issue* pilot devices are actually remediated.
2. On the **next** detection cycle, confirm those devices flip to **Without issue** — this

re-detection is your proof the fix worked and held. Use `PROREM-Report.ps1` to export the detection/remediation success-fail trend to CSV for evidence.

Phase 4 — Expand to production behind guardrails

Step 9. Wave to production

1. Only after the pilot passes (detection accurate, remediation minimal and effective, re-detection clean), create the production assignment. Recommended: **clone** the pilot package to a production copy **named** `PROREM – Service Health – w32time – Production` (or re-assign the same package to the production group — keep pilot and production distinguishable).

1. Assign `PROREM – Service Health – w32time – Production` to **`Remediations – Windows – Production`**, and fill that group in** waves** (a first wave, validate, then widen) rather than the whole fleet at once.

1. Keep the schedule conservative to start (Daily), watch the trend, and widen only when each wave stays clean.

Step 10. Guardrails for production

• **Never widen unproven** — a package only reaches production after a clean pilot.

• **Watch for a remediation loop** — if *Issue remediated* keeps recurring on the same devices every cycle, the fix isn't holding or detection is over-firing; pause the assignment and investigate (Troubleshooting) before it churns the fleet.

• **Keep the ability to reverse** — you can **unassign** a package instantly to stop it; for changes that need active rollback, author a paired "revert" remediation before you deploy the forward one.

• **Change the smallest blast radius** — widen group membership in waves; never assign a new or edited remediation straight to the whole production group.

Phase 5 — Govern the library

Step 11. Register the proven pair in the governed script-pack library

1. Store the final, proven `Detect-ServiceHealth-w32time.ps1` and `Remediate-ServiceHealth-w32time.ps1` in the `Remediations Pack Library` (Requirements §R10) with a **version** (e.g. `v1.0.0`) and a short **changelog**.

1. **Naming convention** — name every pack and its console package the same way so it is identifiable at a glance: `PROREM – <Area> – <Target> – <Stage>` (e.g. `PROREM – Service Health – w32time – Pilot`, `PROREM – Cert Hygiene – ClientAuth – Pilot`). Reuse the exact name across the library, the console package, the assignment, testing, and troubleshooting.

1. **Review & approval** — require a **second admin** to review the detection and remediation source

and to confirm it was proven in **preview + pilot** before the remediation is **approved to act** in production. Record the reviewer and approval date in the changelog.

1. **Reuse** — when the next recurring issue appears, start from an existing proven pack, adapt the detected condition and minimal fix, and run it through the same pilot-first flow. The library is how the practice compounds instead of restarting each time.

5. Testing & Validation — Proactive Remediations at Scale (Detection & Remediation Script Packs for Microsoft Intune)

Validate against the **pilot ring** (`PILOT – Remediations – Devices`) before expanding. Each test has an explicit **PASS/FAIL** gate. Use the included PowerShell scripts (Microsoft Graph) for tenant-side checks and the on-device / console steps for behavior. The worked example is the `PROREM – Service Health – w32time – Pilot` package built in Implementation.

> ***Seed the issue first.*** *On one pilot device, deliberately break the target condition so detection has something to find: set the* ***Windows Time service (****`w32time`****) to Disabled*** *(or stop it). This "seeded issue" device is your positive test; leave at least one pilot device healthy as the negative test.*

Test 1 — Package is present, assigned, and reporting

Procedure: Run `PROREM-Discovery.ps1`, then `PROREM-Validate.ps1`. **PASS:** `PROREM – Service Health – w32time – Pilot` exists, is assigned to `PILOT – Remediations – Devices`, has a schedule, and is returning run status (devices reporting). **FAIL:** Missing / unassigned / no status → fix the assignment and schedule (Implementation §5–6), confirm the Intune Management Extension is healthy (Requirements §R4), then re-run.

Test 2 — Detection correctly flags the seeded issue (and only it)

Procedure: After a detection cycle, open the package → **Device status** in the console. **PASS:** The **seeded** device reports **With issue** (detection exited non-zero) and the **healthy** pilot device reports **Without issue** (detection exited zero). Detection did **not** change either device (it is read-only). **FAIL:** Seeded device shows *Without issue* → detection logic or run context is wrong (Requirements §R6). Healthy device shows *With issue* → detection is over-firing; tighten the condition.

Test 3 — A bad remediation is caught in dry-run (preview) — before it can act

Procedure: With the **preview** remediation assigned (Implementation §6), read the seeded device's remediation **output/log** (`C:\ProgramData\Remediations\`) and the console's pre/post output columns. **PASS:** The preview log shows **exactly the intended minimal change** it *would* make (set `w32time` to Automatic and start it) and **no device was actually changed** — the service is still Disabled after the preview run. If the preview log revealed a wrong or over-broad action, it is caught **here**, with zero fleet impact. **FAIL:** The preview changed the device (it is not truly dry-run), or the log shows an unintended action → fix the remediation before proceeding. **This gate is the safety net; do not skip it.**

Test 4 — Remediation fixes the seeded issue

Procedure: Flip the remediation to its **acting** version (Implementation §8) and let a cycle run. **PASS:** On the seeded device, the remediation reports **Issue remediated** (exit 0); on the device, `w32time` is now **Automatic** and **Running**. The healthy pilot device was **never remediated** (detection kept it out). **FAIL:** *Remediation failed* → check run context/permissions (system vs user, Requirements §R6) and the log. Healthy device got remediated → detection/exit-code contract is wrong (Test 2).

Test 5 — Re-detection proves the fix held

Procedure: Let the **next** detection cycle run; re-read **Device status** and export with `PROREM-Report.ps1`. **PASS:** The previously seeded device now reports **Without issue** on re-detection — evidence the fix worked and **held**. The success/failure trend in the CSV shows "with issue" dropping to zero for the pilot. **FAIL:** The device flips back to *With issue* every cycle → the fix isn't holding (something re- breaks it) or detection over-fires → investigate the loop (Troubleshooting) **before** widening.

Test 6 — Idempotency (no thrash on healthy devices)

Procedure: Over several cycles, watch an already-healthy pilot device. **PASS:** Detection keeps it **Without issue** and remediation **never runs** on it; running the remediation manually on a fixed device makes **no further change** and still exits 0. **FAIL:** Remediation runs repeatedly on a healthy device → detection isn't idempotent or the fix isn't idempotent → correct before Expand.

Acceptance gate

Proceed to **Expand** (production waves) only when **Tests 1–6 pass**: the package is assigned and reporting, **detection flags only the seeded issue**, a **bad remediation would be caught in preview**, the **acting remediation fixes the issue**, **re-detection proves it held**, and the pair is **idempotent** with no thrash. Anything less stays in the pilot ring until it passes.

6. Troubleshooting — Proactive Remediations at Scale (Detection & Remediation Script Packs for Microsoft Intune)

Work top-down: confirm the pair **runs at all** (agent + context), then the **exit-code contract** is read correctly, then that remediation **acts once and stops** (no loop / no over-apply), then **output visibility**, and finally **licensing**. Most issues are a script that ran in the wrong context (so it "did nothing"), or an exit code the console interpreted differently than you intended.

> *Portal paths and column labels below follow the common **Reports** → **Endpoint analytics** → **Remediations** experience and Microsoft Graph **beta** endpoints; both change over time and by cloud — **verify against current Microsoft documentation** for your tenant.*

The script doesn't run at all (no status reported)

- **Intune Management Extension missing/unhealthy.** IME executes the pair on-device; if it isn't installed or is broken, nothing runs and no status returns. Confirm the **Microsoft Intune Management Extension** service is present and running (Requirements §R4); assigning any Intune PowerShell script/Win32 app installs IME.

- **Not assigned / not yet scheduled.** Confirm `PROREM – Service Health – w32time – Pilot` is

assigned to `PILOT – Remediations – Devices` with a schedule (Implementation §6) and allow a cycle. Run `PROREM-Validate.ps1`.

- **Signing/execution policy blocked it.** If your org enforces script signing, an unsigned pack

fails to run. Sign the scripts or set the run/signing options Microsoft documents for your case (Requirements §R5) — **verify current behavior**.

The script runs but "does nothing" (wrong run context)

- **System-vs-user mismatch.** A remediation that edits `HKLM`/a service must run as **system**; one

that edits `HKCU`/the user profile must run in the **logged-on user's** context. The wrong context runs successfully but changes nothing on the intended target. Set **Run this script using the logged-on credentials** to match what the pair touches (Requirements §R6, Implementation §3/§5), and re-test.

- **64-bit vs 32-bit.** A pair touching 64-bit locations must run in **64-bit PowerShell**; check the

package's run-as-64-bit option. **Verify the current option/behavior with Microsoft.**

- **No signed-in user for a user-context pair.** A user-context script has nothing to act on when no

one is logged in — expected; it will act at next run with a user present.

Exit code misread as failure (or success)

- **Detection contract confusion.** Detection must use `exit 0` **= healthy** and **`exit 1` = issue

found**. If your detection exits non-zero on a *healthy* device, the console runs remediation needlessly; if it exits 0 on an *affected* device, remediation never runs. Re-check the exit paths in `Detect-ServiceHealth-w32time.ps1` (or `PROREM-Detect-Template.ps1`).

- **Remediation reporting "failed" wrongly.** A remediation that fixed the device must `exit 0`; a

non-zero exit is reported as *Remediation failed* even if the device is now fine. Ensure the acting path ends in `exit 0` and only genuine failures `exit 1`.

- **Unhandled terminating error.** An uncaught exception exits non-zero and looks like a failure —

wrap risky operations in `try/catch` and set a deliberate exit code (the templates do this).

Remediation loops or over-applies (fix won't hold / churns the fleet)

- **The fix isn't holding.** If the same devices report *Issue remediated* (or recur) every cycle,

something re-breaks the condition after the fix, or the fix addressed a symptom, not the cause. **Pause the assignment**, inspect the on-device log (`C:\ProgramData\Remediations\`), and fix the root cause before re-enabling. `PROREM-Report.ps1` shows the recurring/not-remediated trend.

- **Detection over-fires.** An overly broad detection flags healthy devices, so remediation keeps

acting. Tighten the detection condition to exactly the wrong state.

- **Not idempotent.** A remediation that changes something every run (instead of only when needed)

causes thrash. Confirm the "already healthy → no change → exit 0" path (Testing §Test 6).

• **Guardrail:** you caught none of this at scale if you piloted — this is what the **preview + pilot ring **(Implementation §6/§8) exist for. Unassign to stop instantly; keep a** revert** remediation for impactful changes.

Output is truncated or missing in the console

• **Size/format limits.** The console captures a limited amount of script output; very large output is **truncated**, and reporting is on a **delay**. Keep script output to a short, meaningful line and rely on the **on-device log** (`C:\ProgramData\Remediations\`) for full detail — the templates write both. **Verify current output-size limits against Microsoft's documentation.**

• **Reporting delay mistaken for "no result".** Status can lag the actual run; allow the reporting interval before concluding a device didn't report.

The Remediations experience is unavailable (licensing/entitlement gap)

• **Missing entitlement.** If **Reports** → **Endpoint analytics** → **Remediations** is absent or blocked, the tenant likely lacks the Proactive Remediations entitlement — **do not assume a specific SKU; verify the current licensing requirement with Microsoft** (Requirements §R1) for your subscription and cloud (Commercial vs. GCC/GCC High/DoD may differ).

• **Endpoint Analytics not enabled.** Proactive Remediations lives inside Endpoint Analytics; if analytics isn't onboarded, complete the parent solution first (Requirements §R2).

Useful checkpoints

• **Intune:** Reports → Endpoint analytics → **Remediations** → the package → **Device status** (detection/remediation status + pre/post output); the package **Properties** (run-as context, signing, 64-bit); **Assignments** (groups + schedule).

• **On device:** the **Intune Management Extension** service; the remediation **log** under `C:\ProgramData\Remediations\`.

• **Scripts:** `PROREM-Discovery.ps1` (what's deployed + run-status roll-up), `PROREM-Validate.ps1` (pilot gate), `PROREM-Report.ps1` (success-fail trend / loop detection). All read-only.

7. Best Practices — Proactive Remediations at Scale (Detection & Remediation Script Packs for Microsoft Intune)

Field-proven guidance for running auto-remediation at fleet scale **without** turning it into an auto-outage.

Authoring the pair

• **Detection observes; remediation acts — never blur them.** A detection script must be **read-only

and idempotent**, reaching the same verdict every run and reporting** only** through its exit code (`exit 0` healthy / `exit 1` issue). A detection that "fixes while it checks" hides what the remediation did and breaks the model.

• **Make remediation minimal.** Change the **smallest possible thing** that resolves exactly the detected condition, then stop. No opportunistic extras — every extra action is extra blast radius.

• **Preview before you act — always.** Prove a new remediation in **dry-run/preview** (it logs what it *would* change without changing it) and read the log before you ever let it act. The included remediation template **defaults to preview** for exactly this reason.

• **Honor the exit-code contract.** `exit 0` = success/healthy; `exit 1` = issue/failed. Don't invent other codes to signal state — the console interprets non-zero detection as "run remediation" and reads remediation exit codes as success/fail.

• **Match the run context to the target.** Machine/HKLM/service pairs run as **system**; user-hive/HKCU/profile pairs run in the **logged-on user's** context. The wrong context runs but changes nothing.

• **Log on-device.** Write a timestamped line to a predictable path (e.g. `C:\ProgramData\Remediations\`) so every action has local evidence, independent of console reporting delays and truncation.

Deploy & scope

• **Pilot ring first, every time.** Assign new/edited pairs to `PILOT – Remediations – Devices` and prove them before production — no exceptions.

• **Seed a test issue.** Deliberately break the target condition on one pilot device so you can prove detection catches it and remediation fixes it, and keep a healthy device as the negative test.

• **Widen in waves.** Fill `Remediations – Windows – Production` gradually; never point a new or changed remediation at the entire fleet at once.

• **Keep the schedule conservative.** Start Daily; only tighten cadence when the trend is clean. Faster is not better if a bad fix runs more often.

Operate safely

• **Watch for loops.** If the same devices report *Issue remediated* every cycle, the fix isn't holding or detection is over-firing — **pause the assignment** and investigate before it churns the fleet.

• **Re-detect to prove, don't assume.** Treat the post-remediation *Without issue* result as your evidence; if it doesn't hold, the pair isn't done.

• **Keep an off switch and a reverse.** You can **unassign** a package to stop it instantly; for impactful changes, author a paired **revert** remediation before deploying the forward one.

• **Report on the trend.** Export detection/remediation success-fail regularly with

`PROREM-Report.ps1`; a downward "with issue" trend is proof the practice is working.

Govern the library

- **Name everything by intent.** Use `PROREM – <Area> – <Target> – <Stage>` across the library, the

console package, assignments, testing, and troubleshooting so packs are identifiable and auditable.

- **Version and changelog every pack.** Know exactly which version is live and what changed.
- **Require a second-admin review + approval to act.** A remediation reaches production only after

another admin reviews the source and confirms it was proven in preview + pilot.

- **Reuse proven packs.** Start the next recurring issue from an existing proven pack — the governed

library is how the practice compounds instead of restarting each time.

- **Retire deliberately.** When an issue is designed out or a pack is superseded, unassign and

archive it (with a note) rather than leaving stale automation running.

8. Deliverables Summary — Proactive Remediations at Scale (Detection & Remediation Script Packs for Microsoft Intune)

Everything included in this solution package, and what each piece is for.

Documentation (this guide)

- **Executive Summary** — the business case and outcome in one page.
- **Solution Overview** — the detect/remediate model, why safety is the whole game, the governed

script-pack library concept, how this plugs into Endpoint Analytics (the parent solution), benefits, and honest limitations.

- **Requirements Guide** — every prerequisite (Proactive Remediations licensing, Endpoint Analytics

enabled, enrolled Windows devices, the Intune Management Extension, script-signing/execution posture, run-context permissions, pilot and production groups, a Graph-capable admin workstation, and a governed pack library) with how to **verify** and how to **implement if missing**, plus the category-level Global Requirements. Licensing, feature availability, and behavior are flagged **verify-with-Microsoft**.

- **Implementation Guide** — exact, ordered, click-by-click steps across five phases: author the pair

(idempotent detection + preview-first remediation, correct run context), package and scope, read results and validate, expand to production behind guardrails, and govern the library. Every object is named and reused.

- **Testing & Validation** — six tests with explicit PASS/FAIL gates and an acceptance gate: the

package reports, detection flags a **seeded** issue and only it, a **bad remediation is caught in dry-run**, the acting remediation fixes the issue, **re-detection proves the fix held**, and the pair is idempotent.

- **Troubleshooting** — top-down diagnosis for the script not running / wrong context, exit codes

misread as failure, remediation loops or over-applies, truncated output, and licensing gaps.

- **Best Practices** — authoring the pair, deploy/scope, operate safely, and govern the library.

PowerShell scripts (Microsoft Graph & templates) — `PROREM-Scripts.zip`

Each script has a header explaining **Purpose · How to run · Graph scopes · Expected results**. The three Graph scripts are tenant-side (run from an admin workstation with the Microsoft Graph PowerShell SDK) and **read-only**; the two templates are examples you adapt into your own packs.

1. `PROREM-Discovery.ps1` — read-only inventory of existing remediation script packages, their

assignments, and run status across the tenant.

1. `PROREM-Validate.ps1` — PASS/FAIL check that the pilot package is present, assigned, running on

a schedule, and reporting.

1. `PROREM-Report.ps1` — exports the detection/remediation success-fail trend to CSV for evidence

and reporting.

1. `PROREM-Detect-Template.ps1` — a **read-only detection template**: idempotent, correct

exit-code contract (`exit 0` healthy / `exit 1` issue), makes **no changes**. Your starting point for a detection script.

1. `PROREM-Remediate-Template.ps1` — the one state-changing template: a **remediation template

that DEFAULTS TO a dry-run/preview** (`-WhatIf` / `$DryRun = $true`), demonstrating a safe minimal fix with logging. It says so in its header and refuses to change anything until dry-run is explicitly turned off.

Hero image

- `PROREM-hero.png` — the product image (1200×1200) used on the marketplace listing.

How it's delivered

A **Word** document and **PDF** (cover, table of contents, page numbers, and a scripts appendix), plus all five PowerShell scripts in a single **ZIP** — yours to keep and reuse across **unlimited devices**.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

PROREM-Detect-Template.ps1

Purpose: A copy-and-adapt DETECTION script that models the safe pattern: it OBSERVES ONLY (makes no changes), is IDEMPOTENT (same verdict every run), and reports its result solely through the EXIT-CODE CONTRACT — exit 0 = healthy (do nothing), exit 1 = issue found (run the paired remediation). The worked example checks that the Windows Time service (w32time) is set to Automatic and is Running; replace the CONDITION block with your own read-only check.

How to run: This script is UPLOADED to Intune as the DETECTION script of a remediation package (Reports > Endpoint analytics > Remediations > Create — verify path with Microsoft docs). Intune's management extension runs it on-device on the assignment's schedule. You do not run it manually in production, though you can run it locally on a pilot device to test the verdict. Run context: this example touches a machine service, so package it to run as SYSTEM (Run using logged-on credentials = No). A check of HKCU/user data would instead run in the USER context.

PROREM-Discovery.ps1

Purpose: Lists every Endpoint Analytics Proactive Remediation (device health script) in the tenant with its publisher, run-as context, assignments, and a roll-up of run status (how many devices reported "with issue" vs "without issue", and remediation success/failure). Gives you a one-page picture of what auto-remediation is currently deployed. Makes NO changes.

How to run: Install-Module Microsoft.Graph -Scope CurrentUser # once .\PROREM-Discovery.ps1 .\PROREM-Discovery.ps1 -NameLike 'PROREM' Reads only — makes no changes.

PROREM-Remediate-Template.ps1

Purpose: A copy-and-adapt REMEDIATION script that models the safe pattern: it DEFAULTS TO A DRY-RUN / PREVIEW ($DryRun = $true) so that, until you explicitly disable the preview, it LOGS EXACTLY WHAT IT WOULD CHANGE WITHOUT CHANGING ANYTHING. This is deliberate: prove the intended, minimal change from the log on a pilot device BEFORE you let it act on the fleet. It also demonstrates a MINIMAL fix (change the smallest thing that resolves the detected condition), IDEMPOTENCY (no further change on an already-fixed device), and LOGGING to a predictable on-device path. The worked example sets the Windows Time service (w32time) to Automatic and starts it.

How to run: This script is UPLOADED to Intune as the REMEDIATION script of a remediation package; it runs on-device (via the Intune Management Extension) ONLY on devices whose paired detection exited non-zero. Package it in the SAME run context as its detection (this example = SYSTEM). Pilot flow: upload the DRY-RUN version first, read the preview log, then upload the acting version ($DryRun = $false) once proven.

PROREM-Report.ps1

Purpose: For each Proactive Remediation package (or one named package), exports its run-status roll-up — devices with issue detected, devices remediated, detection failures, and remediation failures — to a CSV so you have evidence of the trend over time and per package. A downward "with issue" count across

successive exports is proof the practice is working. Read-only.

How to run: Install-Module Microsoft.Graph -Scope CurrentUser # once .\PROREM-Report.ps1 .\PROREM-Report.ps1 -PackageName 'PROREM – Service Health – w32time – Pilot' .\PROREM-Report.ps1 -CsvPath C:\Temp\prorem-trend.csv

PROREM-Validate.ps1

Purpose: Validates the acceptance gate for a single Proactive Remediation package (default: 'PROREM – Service Health – w32time – Pilot'): it EXISTS, is ASSIGNED (to at least one group, e.g. 'PILOT – Remediations – Devices'), has a SCHEDULE, and is REPORTING run status back from devices. Produces a clear PASS/FAIL verdict. Read-only.

How to run: Install-Module Microsoft.Graph -Scope CurrentUser # once .\PROREM-Validate.ps1 .\PROREM-Validate.ps1 -PackageName 'PROREM – Service Health – w32time – Pilot'